

Architecture

Hardware, software, and virtual

Architecture

In the context of software development, and in the context of SIA, architecture refers to the overall design and structure of a software system.

This includes the hardware, software, and virtual *components* that make up the system, as well as how they interact with each other.

Hardware

Physical Infrastructure: The servers, storage, and networking equipment that host the system.

- *On-Premise (Ownership)*: Buying physical hardware requires significant upfront capital and internal maintenance but offers total physical control.
- *Cloud Hosting (Acquisition)*: Treating hardware as a service. You acquire raw power from a vendor (AWS, Azure) without owning the physical machines.

High security or niche performance needs may lead to "building" your own data center, while speed and scaling favor "buying" cloud capacity.

Case Study: AWS (Infrastructure as a Service)

AWS allows companies to "acquire" hardware capacity (CPUs, RAM, Storage) through the cloud without owning any physical machines.

A company can build their own server room with cooling, electricity, and security (Ownership), or they can lease virtualized "instances" from AWS.

With AWS, you can "rent" 1,000 servers for one hour and turn them off. This is impossible if you own the physical hardware.

Most companies choose to "buy" their hardware capacity from AWS because managing physical servers is a *Commodity*.

Software

Logical Components: Includes the Operating System, Database Management Systems, and the Application Layer.

- *COTS (Buying)*: Commercial Off-The-Shelf software (e.g., Oracle, Salesforce). These provide industry "best practices" and faster deployment for *commodity* capabilities.
- *Custom Development (Building)*: Writing proprietary code to handle unique business logic.

Modern architecture often "buys" the standard software (OS/DB) and "builds" the unique application features.

Case Study: Supabase

Supabase is an open-source **Backend-as-a-Service (BaaS)** that provides a full software stack (Database, Auth, Storage).

A startup can spend months configuring PostgreSQL, setting up auth, and managing storage buckets, or they can use *Supabase* and start coding features immediately.

By "*buying*" Supabase, the developer accepts Supabase's *predefined structures and APIs*. They gain speed but are now tied to the Supabase ecosystem.

In most apps, the database and auth are *Commodities*. Supabase allows teams to acquire these generic parts so they can focus on their *Differentiator*.

Virtual

The Abstraction Layer: Virtual Machines (VMs), Containers (Docker), and Serverless functions that sit between hardware and software.

- *Portability*: Virtualization allows software to be *independent* of specific hardware. This reduces *Hardware Lock-in*.
- *Managed Abstractions*: You can "buy" higher levels of virtualization (PaaS/SaaS) where the vendor manages everything from the hardware up to the runtime environment.

Using standardized virtual containers makes it easier to switch vendors or move between on-premise and cloud.

Case Study: Docker

Docker is a **Containerization** tool that allows developers to package software with all its dependencies.

It allows you to "*Build Once, Run Anywhere.*"

A developer can "build" a container on their *local* laptop, and "*deploy*" it to an AWS server without changing a single line of code.

If AWS becomes too expensive, the company can *move that same Docker container* to a Google Cloud or Azure server instantly.

Docker provides an abstraction that prevents *Hardware Lock-in*, making the infrastructure part of the system more flexible.

Case Study: Vercel

Vercel is a **Cloud Platform (PaaS)** that provides the infrastructure and deployment pipeline for *web applications*.

Vercel handles all server management, load balancing, and scaling. For the developer, the hardware layer is completely hidden.

By using Vercel, you are "buying" a managed virtual environment. You don't manage the OS or virtualization; you just provide the code.

The high level of convenience comes with a trade-off. Migrating a complex project away from Vercel can be difficult due to proprietary features and pricing structures.